AUTONOMOUS MOBILE ROBOT

KOBUKI PROJECT REPORT

SHAHEER MUSTAFA (A1925543), SAFIN KHAN, HITEN GUPTA
MICROCONTROLLER PROGRAMMING
Adelaide University

Contents

1. Introduction 3
1.1 PROJECT BACKGROUND 3
1.2 PROJECT DESCRIPTION 3
1.3 PROJECT OBJECTIVES AND REQUIREMENTS 3
2. HARDWARE AND SOFTWARE 4
2.1 STM32 MICROCONTROLLER AND KOBUKI ROBOT 4
2.2 SENSORS AND HARDWARE CONNECTIONS 4
2.3 SOFTWARE DEVELOPMENT ENVIRONMENT 4
3. SYSTEM DESIGN 5
3.1 PROGRAM OVERVIEW AND STRUCTURE 5
3.2 SYSTEM FLOWCHART 5
3.3 OVERALL STATE MACHINE (FSM) 6
3.3.1 SEARCH AND ORIENTATION 7
3.3.2 PATHFINDING 7
3.3.3 STORAGE AND DEPOSIT ALIGNMENT 8
4. CODE IMPLEMENTATION 9
4.1 CODE STRUCTURE 9
4.2 KEY FUNCTIONS 10
4.2.1 Provided Functions 10
4.2.2 DISTANCE ODOMETRY 11
4.2.3 ANGLE ODOMETRY 11
4.2.4 ULTRASONIC DISTANCE CONVERSION 11
4.3 UNIQUE FUNCTIONALITY 12
4.3.1 PACKET-PACED FSM 12
4.3.2 MIRRORED LAYOUT CONSTANTS 12
4.3.3 WRAP-SAFE SCAN ANGLE 12
4.3.4 CORRIDOR RECOVERY HELPERS 12
4.3.5 SOFTWARE CLIFF THRESHOLD 12
4.3.6 DEPOSIT ALIGNMENT CODE 12
4.3.7 DISTANCE ADJUSTMENT 13
4.3.8 FAULT HELPERS 13
4.4 ADDITIONAL FUNCTIONALITY 13
4.4.1 LED STATUS FEEDBACK 13
4.4.2 WIN SOUND 14
5. DEVELOPMENT AND TESTING 14
5.1 DEVELOPMENT PROCESS 14

5.1.1 PHYSICAL TESTING....14
5.1.2 SIMULATION TESTING....14
5.2 PROBLEMS ENCOUNTERED AND SOLUTIONS....14
6. RESULTS....16
6.1 FINAL FUNCTIONALITY REVIEW....16
6.2 FINAL TEST RESULTS....16
7. DISCUSSION....16
7.1 ACCURACY....17
7.2 PRECISION....17
7.3 RELIABILITY....17
7.4 VALIDITY....17
7.5 TIMING PERFORMANCE....18
7.6 LIMITATIONS AND POSSIBLE IMPROVEMENTS....19
8. CONCLUSION....20
9. REFERENCES....21
10. APPENDICES....21

Figure 1 – This image shows the layouts, the obstacles and the collection area....3
Figure 2 - Board layout and hardware components....4
Figure 3 – Microcontroller mounted on the Kobuki robot....4
Figure 4 - STM32 NUCLEO-G474RE pinout and connector configuration....4
Figure 5 - System Flowchart - Overview....5
Figure 6 - This diagram shows FSM used to control the Kobuki....6
Figure 7 - FSM – Search and Orientation Phase....7
Figure 8 - FSM – PathFinding Phase....7
Figure 9 - FSM – Storage Phase....8

Table 1 - STM32 hardware connections....4
Table 2 - Files in the project and their responsibilities....9
Table 3 - shows the provided functions and how they were implemented....10
Table 4 - LED 1 Light Feedback....13
Table 5 - Problems and solutions by phase....15
Table 6 - Assessment of the final test against the requirements....16
Table 7 - Results of Final Demonstration....16
Table 8 - Test area specification (dimensions in mm)....18
Table 9 - Limitations of the final design and the improvements that would address them....20

1. INTRODUCTION

1.1 PROJECT BACKGROUND

Autonomous mobile units (AMUs) are self-navigating robots that are increasingly being implemented in parcel sorting operations to move packages between stations. These robots work in close conjunction with humans, which means that the units must reliably navigate with precision and stop safely on command. Incorrect navigation leads to safety issues and possible damage to the robot, parcel or surrounding environment.

The purpose of the project is to utilise an STM32 NUCLEO-G474RE microcontroller to develop odometry and object-detection algorithms that control a Kobuki autonomous mobile robot. A UART link will be used to pass instructions between the microcontroller and Kobuki while an analogue ultrasonic sensor will allow the Kobuki to detect the deposit location and bollards. Visual Studio Code will be used to write a finite state machine (FSM) in C, which permits the robot to identify the open path to the storage unit and precisely navigate towards it.

This report outlines the design, development and testing of the autonomous mobile unit. It begins by describing the hardware components and software environment, followed by an overview of the system architecture, sensor functionality and finite state machine. The implementation of the navigation, obstacle detection and deposit alignment algorithms is then discussed, including the key functions and design decisions. Finally, the report presents the testing results, evaluates the system against the project requirements, and discusses its performance, limitations and potential improvements.

1.2 PROJECT DESCRIPTION

In this project, a Kobuki robot is used to simulate an AMU in a parcel sorting warehouse. Once a parcel has been placed on top of the Kobuki, it must carry the parcel from the collection location to the nearby storage unit. Humans place 2 bollards of length 90 mm to block both ends of one path which guides the AMU along the other. The bollards can be in one of two arrangements (Figure 1). Additionally, the deposit locations can be in one of two positions, D1 or D2. Neither is known before a run. The Kobuki always starts in the same package collection location, but at an arbitrary orientation, and must be able to navigate either layout or deposit the package at either deposit location without human assistance.

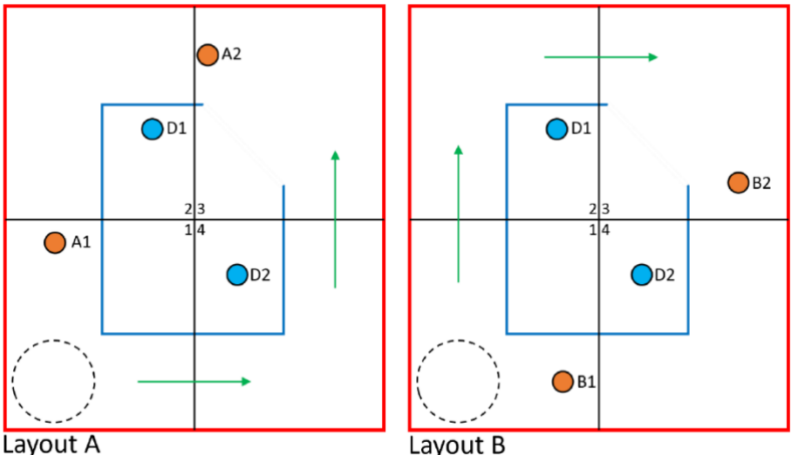


Figure 1 – This image shows the layouts, the obstacles and the collection area

1.3 PROJECT OBJECTIVES AND REQUIREMENTS

The AMU must remain idle until it receives a start signal, then scan its surroundings to identify the layout and navigate through the open path while avoiding bollards. It must locate the deposit position and stop 100 mm away, with the centre of the robot facing the deposit rod, before returning to idle. For safety, the Kobuki must stop and return to idle if B1 is pressed or a wheel drop is detected, while using its bumper and cliff sensors to avoid obstructions and the drainage ledge. The robot must also remain within the

maximum speed of 150 mm/s and enter the first and second sections of the storage route within 30 and 60 seconds, respectively.

The Kobuki is tested on two unknown layouts consecutively using the same program. As a result, it must be able to navigate through any of the four possible combinations of bollard layout and deposit location. The aim of the project is to successfully configure the Kobuki to handle all four combinations of bollard layout and deposit location while meeting the requirements listed above.

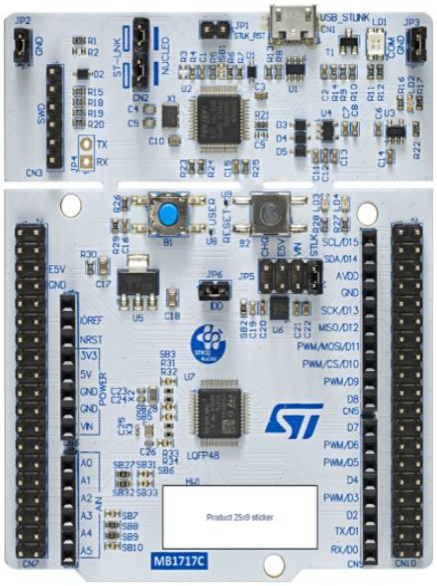


Figure 2 - Board layout and hardware components.

2. HARDWARE AND SOFTWARE

2.1 STM32 Microcontroller and Kobuki Robot

The STM32 NUCLEO-G474RE microcontroller was used to control the Kobuki robot (Figure 2). The microcontroller executes the program and communicates with the Kobuki through UART, sending movement instructions and receiving sensor feedback. The Kobuki provides the motors, wheel encoders, gyroscope, bumper sensors and cliff sensors required for navigation.

2.2 Sensors and Hardware Connections

The STM32 communicates with the Kobuki using UART protocol at a baud rate of 115200. The USART3 interface uses PC10 (TX) and PC11 (RX), with the transmit and receive lines connected between the board and Kobuki. An analogue ultrasonic sensor is connected to PA0 to measure distances to the bollards and deposit rod. The remaining sensors are built into the Kobuki, with their readings received through UART.

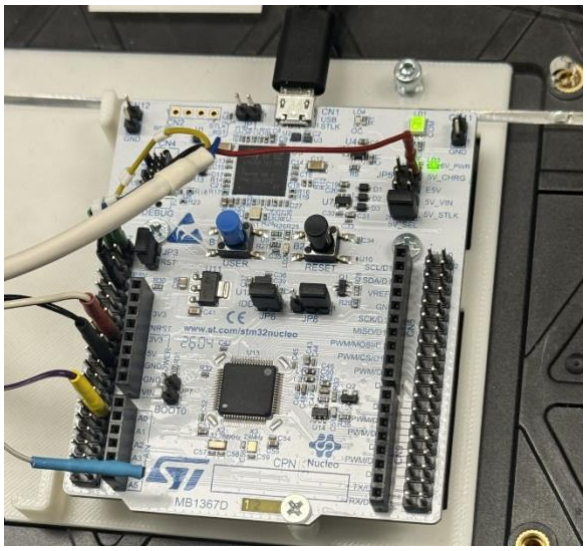


Figure 3 – Microcontroller mounted on the Kobuki robot.

Table X: STM32 hardware connections

STM32 Pin	Function	Connection
PC10	USART3 TX	Kobuki RX
PC11	USART3 RX	Kobuki TX
PA0	ADC1 Channel 1	Ultrasonic sensor output
GND	Common ground	Connected hardware

Table 1 - STM32 hardware connections

2.3 Software Development Environment

Visual Studio Code was used to develop the program in C, with the STM32 toolchain and CMake used to build the firmware. The program was tested on the physical Kobuki robot, while a computer-based simulator was also used to evaluate navigation and identify initial problems.

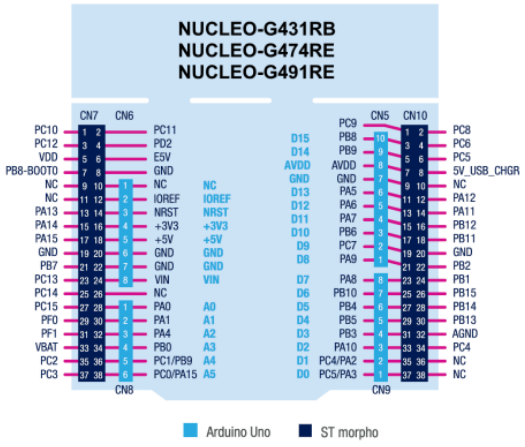


Figure 4 - STM32 NUCLEO-G474RE pinout and connector configuration.

3. SYSTEM DESIGN

3.1 PROGRAM OVERVIEW AND STRUCTURE

The Kobuki robot was programmed as a Finite State Machine (FSM), which means that the Kobuki is in a single state performing one function at a time. A polling super-loop with no RTOS or interrupts is utilised. The main function uses Kobuki_Read() to receive a 50 Hz packet containing the sensor values. This results in a fixed 20 ms control tick, giving deterministic timing and preventing free-run loops. Furthermore, there can only be one transition between states per tick, with a fault check in every state except idle. A fault check returns the Kobuki to the idle state, via a fault state that stops the motors, if B1 was pressed or a wheel drop occurs. In the search, identify, orient and deposit-distance states, a bumper or cliff trigger is also treated as a fault.

A change_to() function was used for state transitions. It has an exit block and an entry block. The exit block was used when leaving the idle state to clear the previous run. The entry block was used to set the motion set points, which were then serviced every tick by the state's function.

The states were grouped into 3 phases: search and orientate, pathfind and storage.

3.2 SYSTEM FLOWCHART

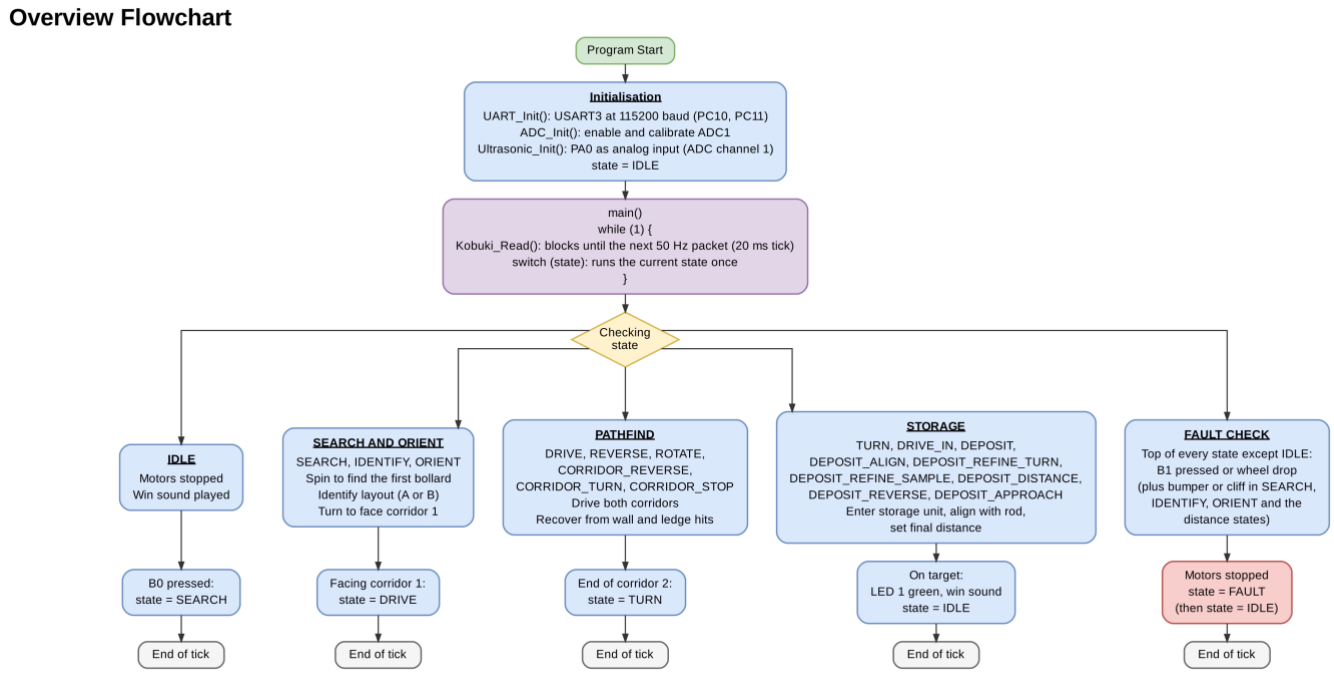


Figure 5 - System Flowchart - Overview

Figure 5 shows the overview flowchart. The program initialises the UART, ADC and ultrasonic sensor, then enters main(), where the current state is checked on every 50 Hz packet. Each state belongs to one of the three phases (search and orientation, Pathfind or storage), so the Kobuki runs that state's actions and then moves to the next state when its condition is met. This takes it through the phases in order until the package is deposited and it returns to IDLE. If a fault is detected, it stops and returns to IDLE. Each path ends the tick and returns to Kobuki_Read(). The flowchart is not described in more detail, since it follows the same logic as the code and the FSM in Section 3.3. The detailed flowcharts of each phase are in **Appendix B.**

3.3 OVERALL STATE MACHINE (FSM)

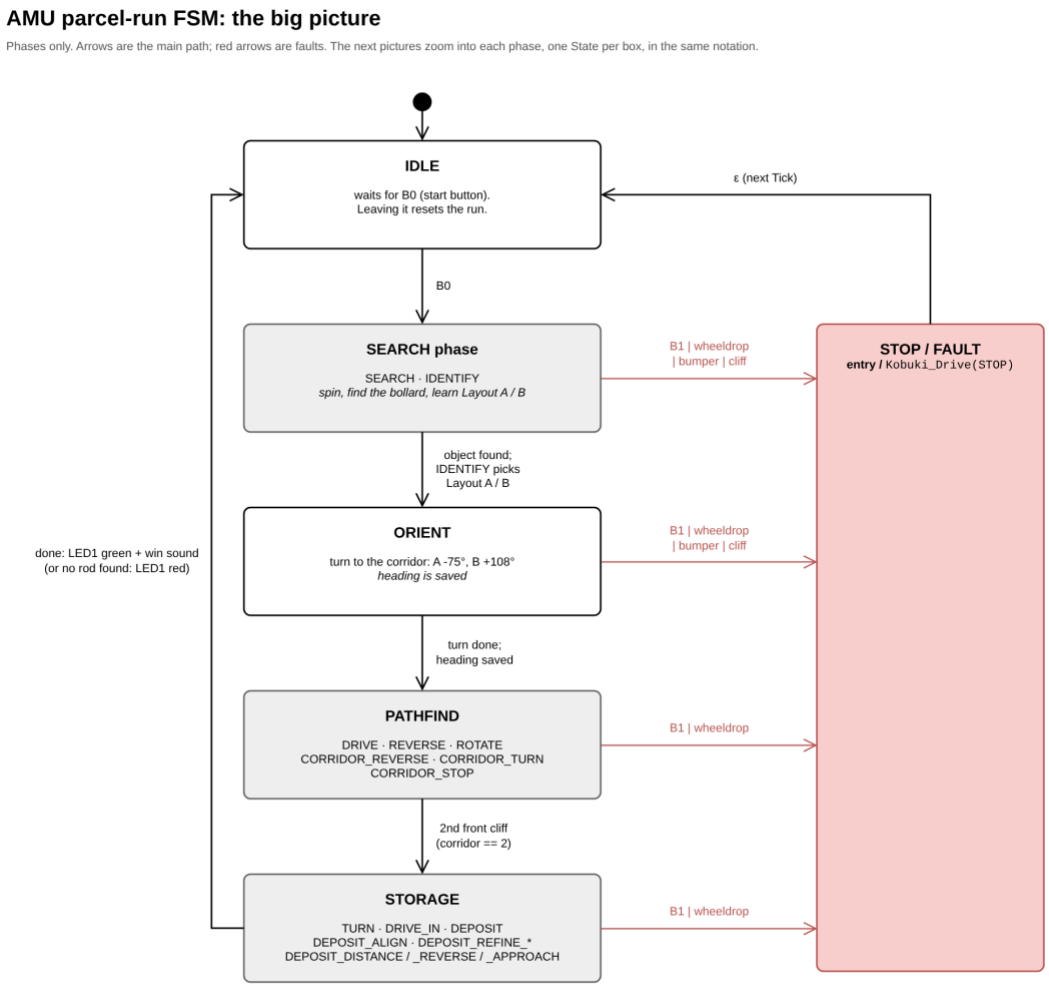


Figure 6 - This diagram shows FSM used to control the Kobuki

The diagram illustrates the main stages of the delivery process, while the individual states and their transitions are shown in the detailed FSM diagrams in **Appendix B** (Figure 5). The FSM consists of 20 states grouped into three main phases: search and orientation, pathfinding, and storage. The IDLE and STOP/FAULT states are kept separate because they do not directly contribute to the delivery process.

The Kobuki begins in IDLE, where it waits for the B0 start button to be pressed. It then enters the SEARCH phase, where it detects the first bollard and identifies Layout A or B. The ORIENT state turns the Kobuki towards the first corridor based on the identified layout. Once the turn is complete, the robot enters PATHFIND, where it navigates through both corridors while recovering from wall and ledge contact. After reaching the end of the second corridor, it enters STORAGE, where it locates the deposit rod, aligns with its centre and adjusts its final stopping distance. Once the target is reached, LED1 turns green, the completion sound plays, and the Kobuki returns to IDLE.

The arrows represent transitions between the different phases, while the red arrows indicate fault conditions. Pressing B1 or detecting a wheel drop causes the robot to enter STOP/FAULT, which stops the motors before returning to IDLE. Bumper and cliff sensors are also used for fault detection in certain states, while in other states they are used for navigation and obstacle recovery. The symbol ε represents an unconditional transition, which occurs on the next control tick.

3.3.1 SEARCH AND ORIENTATION

Object Detection

The Kobuki begins searching when B0 is pressed, rotating 360° anticlockwise at 150 mm/s while the ultrasonic sensor scans for a bollard within 750 mm. If no object is detected after a full rotation, the Kobuki returns to IDLE through the fault state.

Layout Identification

Once a bollard is detected, the Kobuki uses its measured distance to identify the layout. A reading below 575 mm indicates Layout B, while a reading of 575 mm or greater indicates Layout A. LED1 turns yellow for Layout B and red for Layout A.

Initial Orientation

After identifying the layout, the Kobuki rotates 75° clockwise for Layout A or 108° anticlockwise for Layout B to face the first corridor. These angles were determined through physical testing and adjusted until the Kobuki faced the corridor entrance. Once the rotation is complete, the robot enters the pathfinding phase.

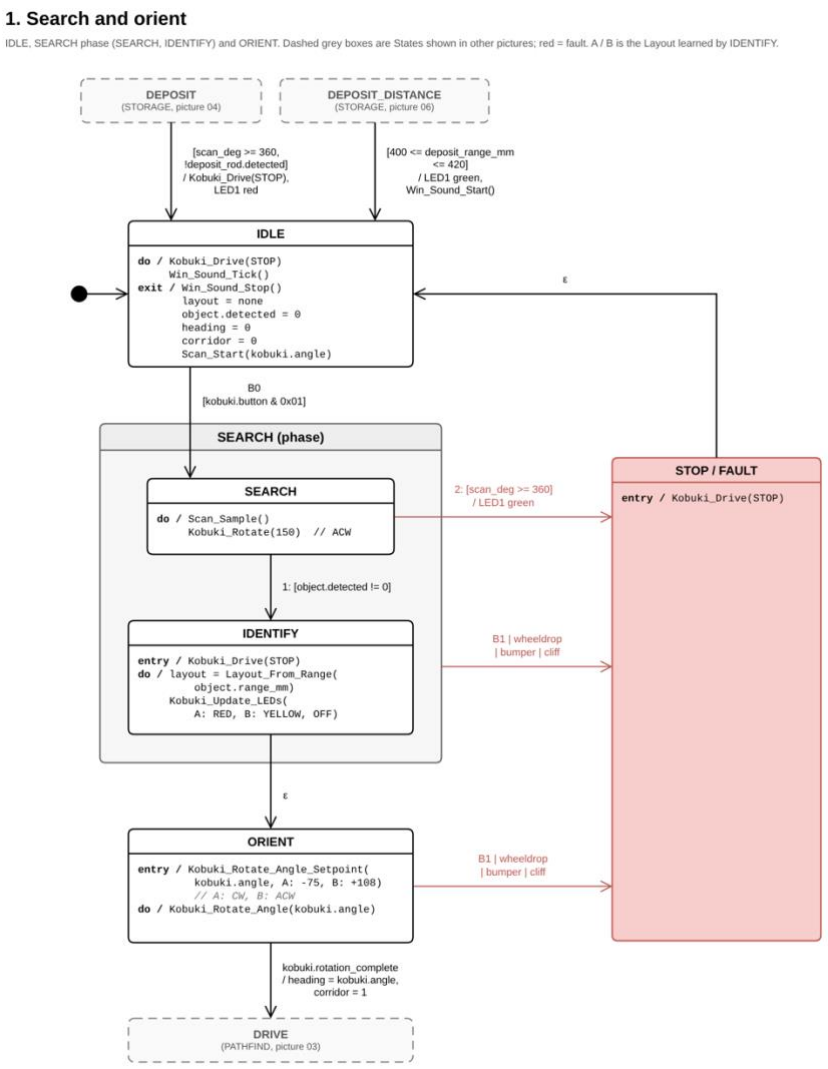


Figure 7 - FSM – Search and Orientation Phase.

3.3.2 PATHFINDING

Corridor Navigation

The Kobuki drives at 150 mm/s until the centre cliff sensor detects the end of a corridor. This was chosen over a fixed driving distance because wheel slip can affect odometry, while the cliff provides a consistent reference point.

At the end of the first corridor, the Kobuki reverses 50 mm and turns 90° towards the second corridor, with the direction determined by the layout. The corridor counter is then updated to 2. When the centre

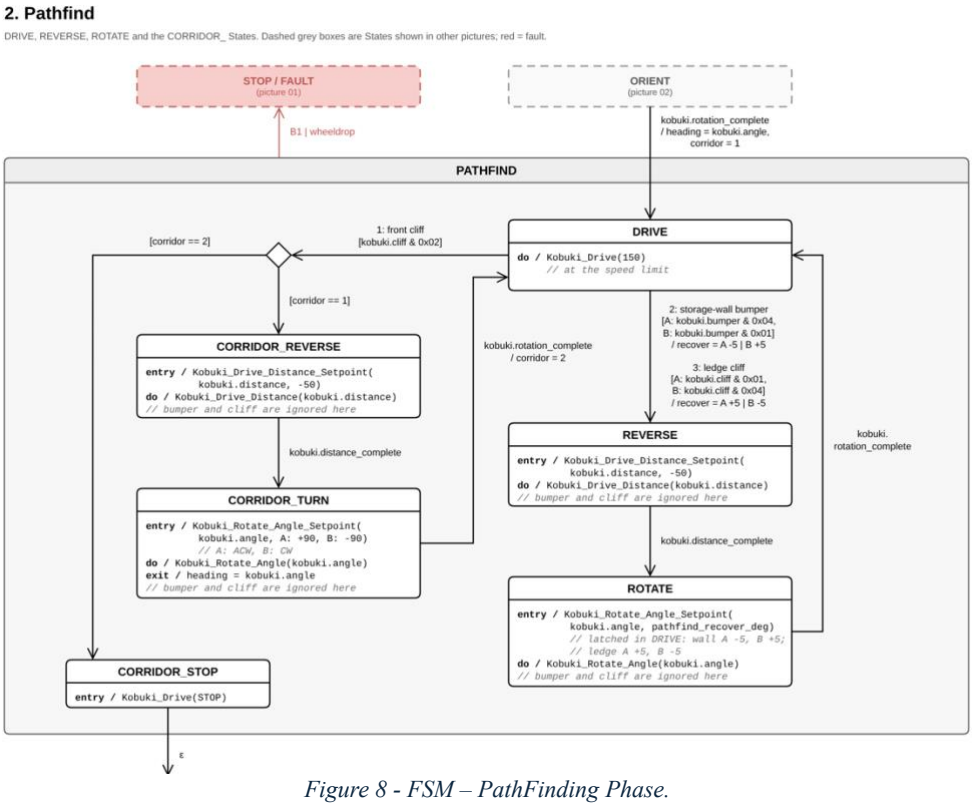


Figure 8 - FSM – PathFinding Phase.

cliff sensor triggers again, the Kobuki stops and enters the storage phase.

Obstacle Detection and Recovery

The Kobuki uses its side bumper and cliff sensors to detect walls and ledges. If either is triggered, it reverses 50 mm and rotates 5° away from the obstacle before continuing forward. The recovery direction depends on the identified layout, allowing the same states to handle obstacles in both configurations.

3.3.3 STORAGE AND DEPOSIT ALIGNMENT

Entering the Storage Unit

The Kobuki turns 131° towards the storage unit, with the direction determined by the layout. It then drives 900 mm or until a bumper is triggered. These values were determined through physical testing to position the robot inside the storage unit with the deposit rod within the ultrasonic sensor's range.

Coarse Alignment

The Kobuki performs a full 360° anticlockwise scan at 50 mm/s, recording the shortest valid ultrasonic distance and its corresponding gyroscope heading. It then rotates towards this heading with a tolerance of 1° to approximately face the deposit rod.

Refined Alignment

The Kobuki performs a ±25° sweep around the detected heading in 2° increments. At each step, it waits 300 ms for the ultrasonic sensor to settle before taking three readings 100 ms apart and selecting the median to reduce measurement noise.

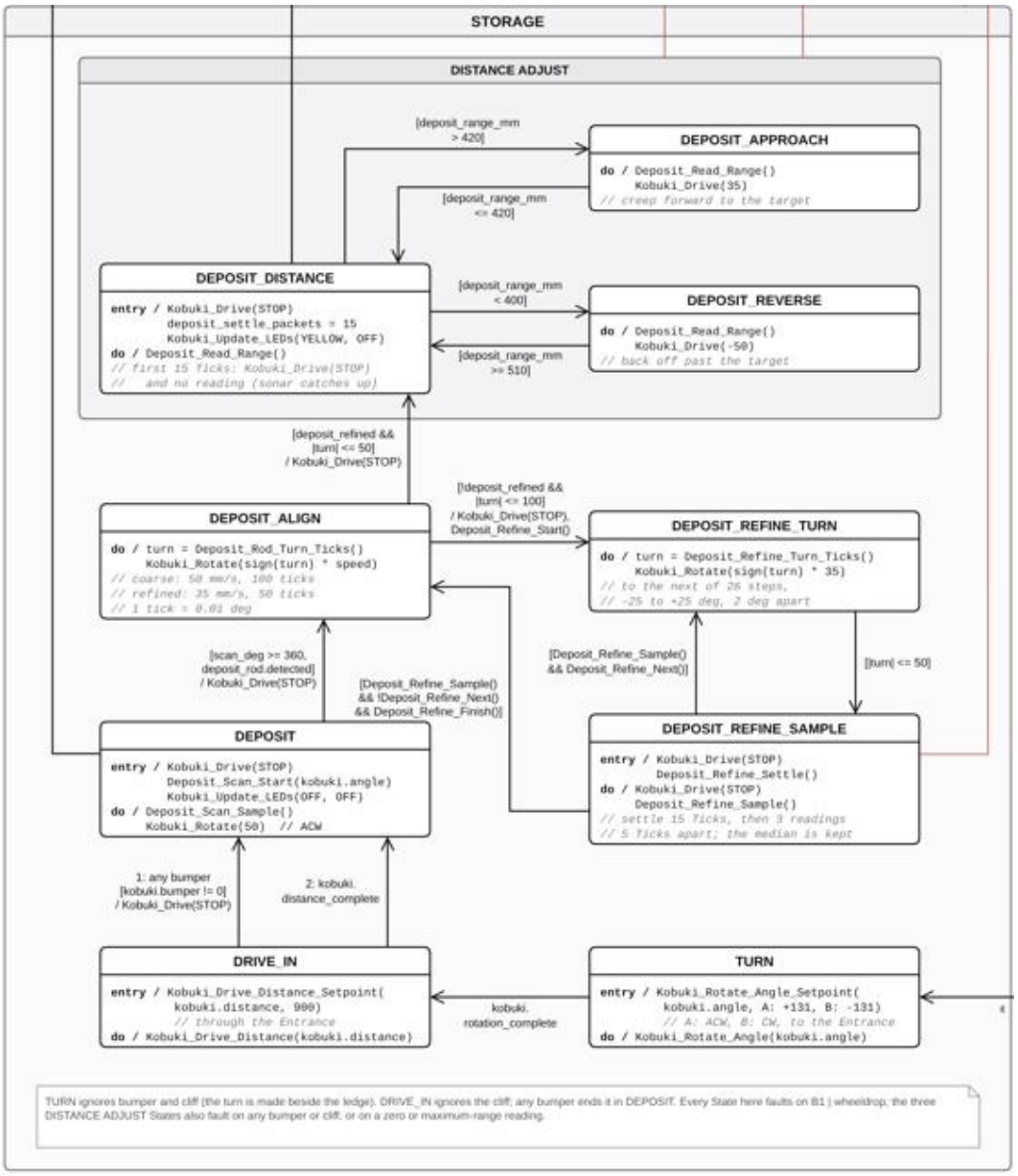


Figure 9 - FSM – Storage Phase

Final Distance Adjustment

The target ultrasonic distance was set to 410 ± 10 mm through physical testing, accounting for the sensor's position on the Kobuki. This corresponds to approximately 100 mm between the robot's front and the deposit rod.

The Kobuki waits 300 ms before measuring the distance. If it is too far, it approaches at 35 mm/s. If it is too close, it reverses at 50 mm/s until reaching 510 mm before approaching again. This ensures the final adjustment is always made moving forward, reducing backlash. Once within the target range, LED1 turns green, the completion sound plays, and the Kobuki returns to IDLE.

4. CODE IMPLEMENTATION

This section of the report focuses on how the FSM was implemented in code. It first shows how the program is organised across files, then lists the key functions that were provided in the practicals and how they were used. It then covers the functionality we developed ourselves, and finally the additional functionality that goes beyond the requirements of the brief.

4.1 CODE STRUCTURE

The program is split into the FSM in main.c, helper files that holds the navigation logic and tuning constants, and three driver layers interact with the hardware. Tabl3 2 lists each key file and the importance.

File	Responsibility	Calls
`main.c`	The `State` enum, `change_to()`, and the main loop with one case per state (the FSM).	helper, Kobuki, UART, ADC, ultrasonic
`Helper/helper.h`	Every tuning constant (speeds, angles, thresholds, tolerances), the `Layout`, `DepositRefine` and `RobotData` types, and the helper declarations.	kobuki.h, ultrasonic.h
`Helper/helper.c`	The shared `robot` data, fault checks, layout selection, scan angle tracking, deposit scan and refine sweep, win sound, and wall and ledge hit handling.	Kobuki, ultrasonic
`Kobuki_Drivers/kobuki.c, .h`	Packet transmit and receive, sensor decoding, drive and rotate commands, the distance and angle controllers, LEDs and the buzzer.	UART
`Ultrasonic/ultrasonic.c, .h`	PA0 setup, conversion of the ADC reading to millimetres, and the detection flag.	ADC
`ADC/adc.c, .h`	ADC1 initialisation and single channel reads.	-
`G474RE_Drivers/uart.c, .h`	USART3 initialisation and byte transmit/receive.	-
`CMakeLists.txt`	Lists the source files and include folders for the build.	-

Table 2 - Files in the project and their responsibilities.

As seen in Table 2, the calls only go downwards. Hardware registers are only accessed in uart.c, adc.c and Ultrasonic_Init(), so register-level code is absent in every other key file. One pass of the main loop works as follows. Kobuki_Read() waits for the next 20 ms block of sensor values and updates them. The switch then runs the current state once. That state may call helper functions, for example Scan_Sample(), which reads the ultrasonic sensor through Ultrasonic_Detect(). Any motor command is sent with Kobuki_Drive() or Kobuki_Rotate(), which pass through Kobuki_Tx() to UART_Tx(). All run-time data is held in one RobotData

structure, robot, declared in helper.h and shared by main.c and helper.c. It holds the latest Kobuki readings, the detected layout, the corridor counter, the scan angle and the deposit scan and refine results.

4.2 KEY FUNCTIONS

4.2.1 Provided Functions

The functions in Table 3 were provided in Practicals 5, 6 or the project resources. They were utilised in the project as provided, but some were adapted to fit the needs of our project.

Function	Provided in	How it was used
`UART_Init`, `UART_Tx`, `UART_Rx`	Practical 5	USART3 at 115200 baud on PC10 and PC11 carries all communication with the Kobuki.
`Kobuki_Tx`, `Kobuki_Rx`	Practical 5	Builds the framed packet with its XOR checksum and finds the 0xAA 0x55 header and verifies the checksum on receive.
`Kobuki_Read`	Practical 5, extended in the project resources	This function was called once in the beginning of every loop. It updates bumper, wheel drop, cliff, button, distance and angle (if checksum passes). We extended it to read the raw cliff values.
`Kobuki_Drive`, `Kobuki_Rotate`	Practical 5	Every speed (driving and rotating) command passes through these, so the 150 mm/s limit is applied in one place.
`Kobuki_Rotate_Angle_Setpoint`, `Kobuki_Rotate_Angle`	Project resources	Kobuki_Rotate_Angle_Setpoint is called once on entry to each turning state (orient, recovery rotate, corner turn, storage turn) to record the target heading. Kobuki_Rotate_Angle is then called every tick in that state, driving the turn towards the target until it sets rotation_complete to be true.
`Kobuki_Drive_Distance_Setpoint`, `Kobuki_Drive_Distance`	Project resources	The rotate and drive functions use the same logic. These functions were used for the 50 mm reverses (recovery) and the 900 mm drive-in to storage area.
`Kobuki_Update_LEDs`	Project resources	Used for the status LEDs.
`ADC_Init`, `ADC_Read`	Practical 6	Configure and read ADC1 channel 1 (PA0) for the ultrasonic sensor.
`Ultrasonic_Init`, `Ultrasonic_Read`, `Ultrasonic_Detect`	Practical 6	Set up PA0, convert the ADC reading to millimetres and set the detected flag. We changed the detection range from 700 mm to 750 mm.
Switch-case FSM	Practical 6	We kept the idea of one case per state, but restructured the loop (as described in 4.3.1).

Table 3 - shows the provided functions and how they were implemented.

Every motion command, including the deposit distance and angle adjust, ends in Kobuki_Drive() or Kobuki_Rotate(), which limits the speed (**A1**).

The setpoint functions, used for states involving fixed movements, record the setpoint once in the state's entry block, and the matching controller (Kobuki_Drive_Distance or Kobuki_Rotate_Angle) is called every tick until it sets the completion flag and the state exits. The reverse state is an example (**A2**).

4.2.2 DISTANCE ODOMETRY

The Kobuki signals distance in encoder ticks. The radius of the wheel is r = 35 mm and one tick is 132/(6545 × 52) of a wheel revolution, so one tick can be calculated as follows:

$$dtick = \frac{2\pi(35) \times 132}{(6545 \times 52)} \approx 0.0853\ mm/tick$$

As a result, the distance can be found using the following formula $d = \Delta N \times dtick = \Delta N \times 0.0853$, so a d =50 mm reverse yields 586 ticks. `Kobuki_Drive_Distance_Setpoint()` converts the requested distance to ticks and stores the target. Each tick, `Kobuki_Drive_Distance()` finds the remaining distance e_d and commands a speed using the following procedure.

$$ed = (Nset - Nnow) \times dtick$$

$$v = sgn(ed) \times 50 + 1.0 \times ed$$

The minimum speed (MIN_SPEED, 50 mm/s) keeps the robot moving until it reaches the target, and Kobuki_Drive() limits the final speed to ±150 mm/s. The distance counter is 16-bit and wraps at 65535 ticks, so the function also compares against the wrapped target when a movement crosses this boundary. The program considers the movement complete when the calculated error e_d is <2 mm. At 50 mm/s the robot moves about 1 mm per 20 ms packet, which is smaller than the 2 mm window, so it cannot skip past it.

4.2.3 ANGLE ODOMETRY

The Kobuki reports its angle in units of 0.01°, from −18000 to +18000 ticks (−180° to +180°):

$$\theta = \frac{N\theta}{100}$$

Kobuki_Rotate_Angle_Setpoint() shifts the current reading by +180° into the range 0° to 360°, adds the requested turn, and stores the result as the target heading, wrapped into the same range. Each tick, Kobuki_Rotate_Angle() finds the heading error e_θ (to the nearest whole degrees) and folds any error above 180° back to the shorter way round, for example 170° to −170° is 20° anticlockwise, not 340° clockwise. It then commands:

$$v = sgn(e_\theta) \times 50 + (1 / 0.9) \times e_\theta$$

`Kobuki_Rotate()` limits the result to ±150 mm/s. The turn is complete when the error is 0°, so the heading is accurate to around one whole degree.

4.2.4 ULTRASONIC DISTANCE CONVERSION

The ultrasonic conversion from Practical 6 is unchanged and it scales the 12-bit reading with integer arithmetic, so no floating point is needed (A3). The ultrasonic sensor outputs an analogue voltage proportional to range. Next, the 12-bit ADC (0 to 4095) converts this to a reading. `ultrasonic.c` used a full-scale range of 5120 mm which means that the distance is:

$$d = ADC_{reading} \times \frac{5120}{4095}$$

The resolution is $5120/4095 \approx 1.25$ mm per count. This is the resolution of the software conversion and not a reflection of the guaranteed accuracy of the sensor. The `Ultrasonic_Detect()` function then compares the measurement against the 750 mm detection threshold to determine whether an object has been detected. The ADC is configured in `adc.c` before the robot begins operating. The initialisation procedure enables the ADC clock, activates the voltage regulator and performs ADC calibration. During operation, `ADC_Read()` selects the required input channel, starts a conversion and waits until the result is available.

4.3 UNIQUE FUNCTIONALITY

The following was developed by us and is not part of Practicals 5 and 6 or the project resources. The behaviour of each item is described in Section 3, so this section covers how it was built in code.

4.3.1 PACKET-PACED FSM

Practical 6 used two switch statements, one for transitions and one for motor outputs, with blocking delay loops such as AvoidDelay(). We replaced this with a single switch in which each case holds its own transitions and motor command. There were also no state blocks involved because Kobuki_Read() repeats the loop at 20 ms. Every wait is instead a counter decremented once per tick. This was used in the 15-packet settle delay and the timing of the win sound.

4.3.2 MIRRORED LAYOUT CONSTANTS

Layout_From_Range() applies the 575 mm differentiation criteria once and the result is stored in robot.layout. Layout_Pick() then returns one of two values depending on that layout, which lets the two mirrored layouts share every state. The orient, corner, recovery and storage turns are each defined in helper.h as an A and B pair, and the wall bumper and ledge cliff sides are chosen the same way. Each value can therefore be changed in one place, and the state logic is not duplicated for each layout (**A5**).

4.3.3 WRAP-SAFE SCAN ANGLE

The gyro angle wraps at ±18000 ticks (±180°), so a full 360° spin cannot be measured by comparing the start and current angle. Scan_Update_Turn() instead adds the change in angle on every tick, folding any jump of more than half a turn back, so scan_ticks is the total turn. Scan_Gave_Up() becomes true at 360°, and the same counter serves both the search and the deposit scan. The static helpers angle_wrap() and angle_turn() apply the same folding to find the shortest turn to a saved heading (**A6**).

4.3.4 CORRIDOR RECOVERY HELPERS

Corridor_Take_Wall_Hit() and Corridor_Take_Ledge_Hit() each test the sensor on the wall or ledge side for the stored layout. When the sensor triggers, the helper also stores the turn away from the hazard in pathfind_recover_deg. REVERSE and ROTATE then use this stored value, so the same two states recover from both a wall and a ledge. The ledge turn is the negative of the wall turn, so the Kobuki always turns away from what it hit. REVERSE and ROTATE use the latched value, which means they are shared by both hazards and both corridors. DRIVE is used for both corridors. When the front cliff triggers, robot.corridor determines the next state: CORRIDOR_REVERSE (back off, then turn the corner) in corridor 1, or CORRIDOR_STOP (stop, then start the storage phase) in corridor 2 (**A7**).

4.3.5 SOFTWARE CLIFF THRESHOLD

The Kobuki's own cliff flag uses a fixed threshold that cannot be calibrated to the test area. Kobuki_Read() was therefore extended to read the raw cliff values (sub-payload header 0x05), which raised SIZE_FEEDBACK to 39 bytes, and store them in cliff_raw. A sensor counts as seeing the ledge when its reading is below CLIFF_RAW_LIMIT (1100), and its bit is set in the cliff byte. Kobuki_IsCliffDetected() checks these bits by side (**A8**).

4.3.6 DEPOSIT ALIGNMENT CODE

A single state, DEPOSIT_ALIGN, is used for both alignments. The flag deposit_refined selects the tolerance (1° or 0.5°) and turn speed (50 or 35 mm/s) and decides whether the state exits to the refine sweep or to DEPOSIT_DISTANCE.

Deposit_Scan_Sample() keeps the shortest valid range and its gyro heading. It ignores zero and anything beyond DEPOSIT_SCAN_MAX_RANGE_MM (3000 mm), so a wall or bollard farther away than the rod cannot become the heading (**A9**). In the sweep, the median and the measured gyro heading of each of the 26 steps are stored in the DepositRefine structure (**A10**). Deposit_Refine_Finish() finds the shortest range, groups neighbouring steps within 10 mm of it, and aims at the midpoint of the widest group, using the wrap-safe helpers to average the first and last headings. If two groups are equally wide, the one nearer the coarse heading is used.

4.3.7 DISTANCE ADJUSTMENT

DEPOSIT_DISTANCE is the only state that decides the next move. In DEPOSIT_REVERSE and DEPOSIT_APPROACH, the Kobuki drives only until the range reaches the end condition, then returns to DEPOSIT_DISTANCE, so the tolerance check is in one place (**A11**). Its settle counter is set in the entry block and counted down each tick. Deposit_Read_Range() returns false for a zero or maximum-range reading, and the three distance states then go to FAULT.

4.3.8 FAULT HELPERS

Fault_B1_Wheeldrop() is checked in every state except IDLE. Fault_Any(), in addition, checks the bumpers and cliff sensors. It is used in SEARCH, IDENTIFY, ORIENT and the three distance states. All other states check only Fault_B1_Wheeldrop(), since contact is expected in the corridors and drive-in, and the deposit scan and refine states only turn on the spot (**A12**).

4.4 ADDITIONAL FUNCTIONALITY

The features in this section are not required by the brief. They were added to make the robot easier to test, debug and/or demonstrate.

4.4.1 LED STATUS FEEDBACK

Kobuki_Update_LEDs() was provided in the project resources. The LED colour was configured to show the Kobuki decision-making and also how a run ended (without needing a debugger), so a failed run could be told apart from a successful one at a glance. This is summarized in the table below.
Table X: Meaning of LED 1.

Event	LED 1
Layout A identified	Red
Layout B identified	Yellow
Deposit scan starts	Off
Adjusting the distance to the rod	Yellow
Stopped on target	Green
No echo found in the deposit scan or refine sweep, or the range reading was lost during distance adjustment	Red
Search found nothing in a full turn	Green

Table 4 - LED 1 Light Feedback.

4.4.2 WIN SOUND

When the Kobuki stops on target, it plays a fanfare of about 3 seconds on the buzzer, which signals to the person unloading the package that the run has finished. Kobuki_Play_Note() converts a frequency into the Kobuki's note value (363636 divided by the frequency), and the notes are stored as a table of 16 entries in helper.c. The Kobuki plays at most 255 ms per note, so each long note is several entries of the same pitch. Win_Sound_Tick() is called once per tick in IDLE and sends the next note only when its turn comes. Every duration and pause is a multiple of the 20 ms packet, so each note starts on a packet, and the control tick is never delayed. Leaving IDLE calls Win_Sound_Stop(), so starting a new run cuts the sound off (**A13**).

5. DEVELOPMENT AND TESTING

5.1 Development Process

The development process alternated between testing on the physical track and testing in a PC simulation developed by a student. Constants were first estimated or calculated, then checked in the simulation and finally corrected on the Kobuki.

5.1.1 Physical Testing and Development

Many constants in `helper.h` depend on the real Kobuki, the sensor mounting and the test area, so they had to be physically measured. The sensor positions relative to the front of the Kobuki had to be measured for the final distance adjustment to be 100mm since the code required a value measured from the ultrasonic sensor to the deposit rod. Additionally, the range at which the first bollard was detected in each layout was measured for layout identification purposes. Moreover, the stopping distance from the rod was measured with a tape measure across trials to test the accuracy of the deposit.

Some values could not be calculated and were found by trial and error. This included the orient turns (75° and 108°), the storage turn (131°) and drive-in distance (900 mm), the 50 mm and 5° recovery movements, and the raw cliff limit (1100). Each was changed one at a time and run repeatedly until the behaviour was consistent.

5.1.2 Simulation Testing

A PC simulator was used to test the firmware. It compiles the same `Src` files unchanged and runs them against a simulated Kobuki (50 Hz packets, gyro, encoders, bumpers, cliff sensors and wheel drop), a 10 Hz ultrasonic sensor with 100 ms latency, and the project arena. Each run is scored against the requirements in Table 1 and produces a replay that can be stepped through. It was used to do the following:

- run all four layout and deposit combinations from multiple starting angles.
- test the safety cases, such as pressing `B1` or lifting the robot.

This let logic errors, such as wrong state transitions and gyro wrap-around, be found and rectified without physical robot testing.

The simulator was not accurate enough to replace physical testing. It had modelling assumptions including the ultrasonic beam being a fixed cone (with an assumed 10° half-angle). Additionally, the wheel slip was approximated, noise was bounded. Furthermore, it lacked real-world errors such as loose mounting, floor surface or battery effects, and no real echo behaviour. Moreover, its tolerance for the deposit checks (25 mm and 5°) were too high in retrospect. For example, at one point we got the simulation to run perfectly 36/36, but when testing in the real arena, it would not even correctly detect the first bollard. In contrast, the final code yielded a much worse simulation assessment, yet it worked flawlessly in real-life. This highlights that the

simulation should be used as a tool and not overly relied on, especially considering the final test was on the physical board.

5.2 Problems Encountered and Solutions

Table 5 lists the main problems in each phase and how they were solved. The deposit state was the biggest obstacle and is covered more extensively.

Phase	Problem	Solution
Search and orient	The first bollard was not always detected within the 700 mm range	Detection range raised to 750 mm
Search and orient	The turn that faces the corridor differs between the layouts	A separate constant for each layout, chosen with `Layout_Pick()` and tuned by trial and error
Pathfind	Driving a fixed distance drifted because wheel slip accumulates error	Drive until the centre cliff sensor detects the end of the corridor
Pathfind	The Kobuki's cliff flag uses a fixed threshold that cannot be tuned to the test area.	Read the raw cliff values and apply `CLIFF_RAW_LIMIT` (1100)
Pathfind	Repeated wall and ledge contacts	Reverse 50 mm and rotate 5° away from the contact, using the shared REVERSE and ROTATE states
All	The gyro angle wraps at ±180°, which broke the 360° scans and the shortest-turn calculation	Wrap-safe helpers (`Scan_Update_Turn()`, `angle_wrap()`, `angle_turn()`)
Storage	A poor drive-in position left the rod out of the sensor's range	Storage turns and drive-in distance was found by trial and error until the constants yielded consistent results.

Table 5 - Problems and solutions by phase.

5.2.1 Deposit Alignment and Distance

Alignment and stopping distance were the hardest parts to make accurate because the requirement had low tolerances, and the sensor only updates at 10 Hz which leaves room for noisy echoes. The early versions used a single and much faster scan. The sensor output had not caught up after turning, so the saved heading was offset and the Kobuki often finished misaligned and/or at the wrong distance. This was fixed by slowing the scan, adding the refine sweep with median readings and a settling delay, and always making the final distance correction moving forward. All these protocols ensured that the final alignment and distance would be consistent and accurate.

5.2.2 Bollard Detected Instead of the Deposit Rod

When the closest bollard was in the same direction as the deposit rod, the scan sometimes detected the bollard as the target instead of the rod which gave a wrong coarse heading. The deposit scan originally accepted any echo up to the sensor's maximum range of 5000 mm (`ULTRASONIC_MAX_RANGE_MM`). This was fixed by adding a cutoff in `Deposit_Scan_Sample()`, `DEPOSIT_SCAN_MAX_RANGE_MM` which was set to 3000 mm. It is set above the rod's range from the drive-in stop and below the nearest other target, so only the rod is accepted.

6. RESULTS

6.1 FINAL FUNCTIONALITY REVIEW

No.	Requirement	Result	Evidence
1	Stay idle until it receives a start signal, then scan its surroundings.	Met	The Kobuki stayed in IDLE until B0 was pressed, then began its scan.
2	Identify the layout, avoid the bollards and take the open path to the storage unit.	Met	The Kobuki identified the layout from the first bollard it detected and took the open path to the storage unit without hitting a bollard.
3	Find the deposit location and stop 100 mm from it, with the centre of the robot facing it.	Met	The coarse-to-fine alignment found the deposit rod and the distance adjustment stopped the Kobuki 100 mm ($\pm 5mm$) from it, with the centre of the robot facing the rod.
4	Return to idle after depositing the package.	Met	Once the Kobuki was on target it returned to IDLE.
5	Stop and return to idle if B1 is pressed or the robot is lifted off the ground (wheel drop).	Met	Pressing B1 or lifting the robot returned it to IDLE from every state, and the Kobuki passed the safety checks.
6	Use the bumper and cliff sensors as fail-safes against obstructions and the drainage ledge.	Met	The Kobuki recovered from wall and ledge contact in the corridors, and stopped on any bumper or cliff trigger in the search, orient and deposit-distance states.
7	Never exceed 150 mm/s.	Met	The speed never exceeded 150 mm/s.
8.1	Enter the first section within 30 seconds.	Met	Met in both configurations (see Table 7).
8.2	Enter the second section within 60 seconds.	Met	Met in both configurations (see Table 7).

Table 6 - Assessment of the final test against the requirements.

6.2 FINAL TEST RESULTS

Configuration	First section (s)	Second section (s)	Distance from rod (mm)	Total time (s)
Layout A with Deposit 2	26	20	106	116
Layout B with Deposit 2	27	20	102	117

Table 7 - Results of Final Demonstration

7. DISCUSSION

Table 6 assesses the final test against each of the requirements listed in the project description. Table 7 gives the times and distances measured in the final test for each of the two configurations. These results are used to

analyse the final test results in terms of accuracy, precision, reliability and validity. Additionally, the timing performance, limitations of the experiment and future improvements that can be made will be discussed.

The Kobuki functioned with great success in the final test, completing both configurations back-to-back with the same program, as shown in the submitted video. The aim of the project, to successfully configure the Kobuki to handle all four combinations of bollard layout and deposit location while meeting the requirements (Table 1), was met.

7.1 ACCURACY

The Kobuki accurately detected the bollards and chose the right layout path in both configurations. The Kobuki did not follow a perfectly straight trajectory along the corridors and needed around 6 to 8 wall or ledge contacts per run. In the final test it made 8 and 7 contacts in the two configurations. A recovery protocol followed each contact, so the Kobuki still completed the corridors. The design relied on the recovery and not on a straight trajectory. This was a compromise made to prioritise consistency in delivery over path accuracy. In the final test the Kobuki stopped 106 mm and 102 mm from the rod, which is 6% and 2% above the target of 100 mm. Additionally, the centre of the Kobuki was visibly in line with the centre of the deposit rod for both configurations. This shows that the coarse-to-fine alignment pointed the Kobuki at the rod accurately enough to meet the requirement.

7.2 PRECISION

Precision describes how repeatable the Kobuki's behaviour was over multiple runs. The same starting orientation/position and layout configuration yielded a near-identical Kobuki path from run to run, which shows high precision. Differences in the starting orientation/position led to some variation in the path, but the overall precision was still high.

In the final test the Kobuki stopped 106 mm and 102 mm from the rod, a spread of 4 mm, which is well inside the ±10 mm margin of the distance controller. The spread was minimised by the 15-tick settle before each distance reading, so the sensor output is stable when it is read. Additionally, the final correction is always made moving forward, which avoids backlash. The alignment was equally consistent. The refine sweep uses the median of three distance readings at each step, so a single noisy echo does not skew the refined heading, and the 0.5° refined heading tolerance corresponds to a sideways offset of about 3.6 mm at 410 mm.

The path along the corridors also had some common patterns. The contacts almost always happened at the very start of the run, straight after the orientation state. Additionally, once the Kobuki made contact, it made multiple contacts in a row before it was clear of the wall/ledge.

7.3 RELIABILITY

The final implementation of the code was very reliable at every phase of the delivery. This includes the deposit phase which was previously somewhat unreliable. Each four combinations of bollard layout and deposit location was tested 2 times before the final test and the Kobuki scored 8 of 8 for all runs. Various starting angles and positions (different centering) were also selected by random to ensure that the Kobuki's functionality worked for all starting positions. These results were reflected by the final test running 2 for 2.

7.4 VALIDITY

Validity refers to how well the tests assess whether the Kobuki meets the requirements. The final test was conducted on a track built to dimensions very close to those specified in the brief (Figure xy), with the same bollard positioning and the same sensors and hardware, so the results are directly relevant to the requirements.

One factor affecting the validity is that the final test was only two runs, so it shows that the Kobuki met the requirements in those two configurations. However, there are four possible configurations of layout and deposit location, combined with the many possible starting orientations/positions of the Kobuki, so a failure in a case that was not tested would not have been seen. Increasing the number of tests would give a better estimate of how reliable the Kobuki is across all of these cases. Moreover, the alignment with the rod was judged by eye, so it is subjective, could be judged differently by another observer, and cannot give an exact offset. Additionally, the stopping distance was measured with a tape measure to ±1 mm, but realistically the uncertainty was larger, at about ±2.5 mm, because the 1 mm markings were hard to read.

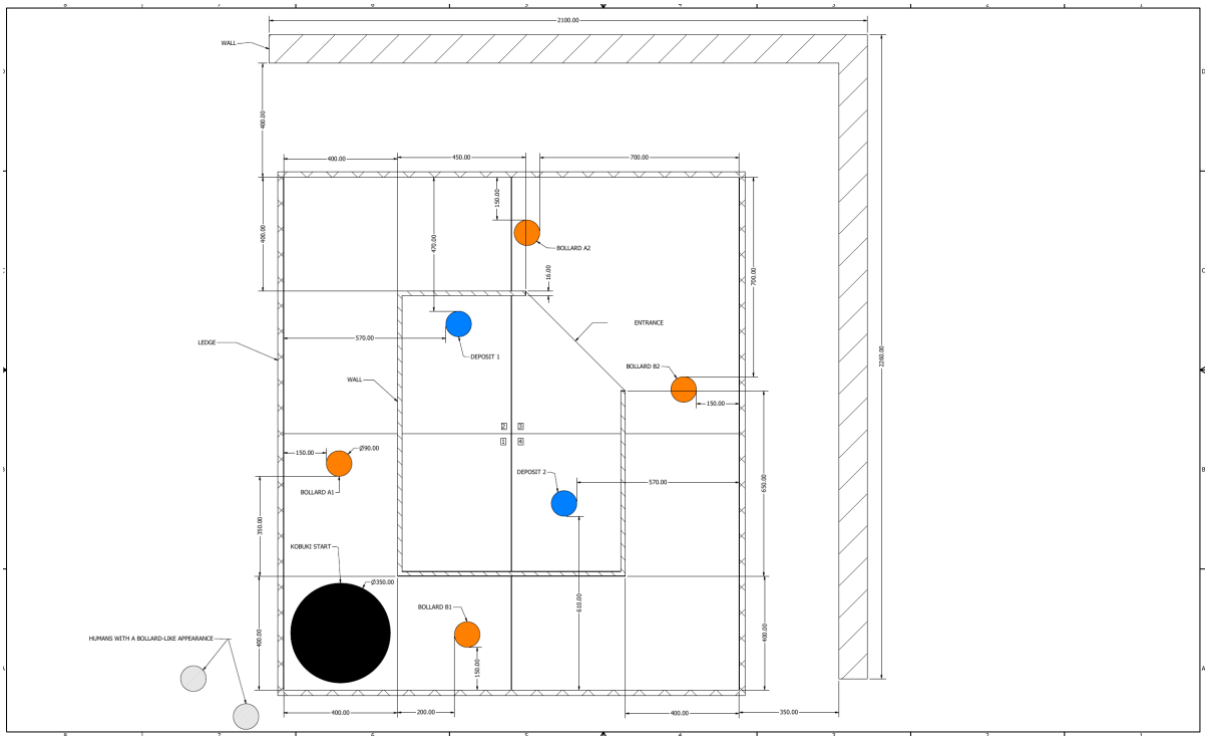


Table 8 - Test area specification (dimensions in mm).

7.5 TIMING PERFORMANCE

In the final test the Kobuki took 116 seconds and 117 seconds to deposit the package in the two configurations. The individual timings for each section are shown in Table 7. The time taken for the recovery from wall/ledge contacts contributes heavily towards the time, but the timing requirements are still met. The storage phase also takes a significant part of the run, because it includes a full 360° scan at 50 mm/s and a 26-step refine sweep with a 15-tick (300 ms) settling delay at every step. This was a compromise of speed for accuracy and precision, made to meet the alignment requirement.

7.6 LIMITATIONS AND POSSIBLE IMPROVEMENTS

No.	Limitation	Possible improvement
1	The main limitation was the path along the corridors. The Kobuki needed 6 to 8 wall or ledge contacts per run, so it excessively relied on the recovery. All the contacts tended to happen consecutively in groups. For instance, the Kobuki would have 3 or 4 contacts consecutively before full recovery. Additionally, the majority of the contacts happened straight after the orientation state, and the orient turn angles (75° and 108°) are fixed constants, so any error in the heading when the bollard was detected carries into the corridor.	Increasing the correction angle, moving back further, or a combination of both would reduce the consecutive contacts. Since most contacts happened one after another, the current 50 mm reverse and 5° rotation was often not enough to clear the wall or ledge in a single recovery. Fewer contacts would reduce the time lost to recovery and improve the accuracy of the path. The orient turn angles could also be adjusted, as the Kobuki would always hit the wall initially, often immediately after orienting for Layout A or B. Alternatively, the searching phase could be slowed so the bollard detection could be more consistent. Despite the slowing, this could lead to reduced contacts which would in turn decrease the overall time. Since the same contacts happened in every run, this offset was consistent, so correcting it would improve accuracy and reliability.
2	The ultrasonic sensor position affected the distance measured during the initial bollard scan. Since layout identification uses a fixed distance threshold, changes in the sensor position could cause the Kobuki to select the incorrect layout and follow the wrong path. Additionally, the final deposit distance is measured from the Kobuki's body, but the distance controller works from the ultrasonic sensor, so the sensor position affects the final deposit distance considerably. The sensor was also not rigidly fixed to the Kobuki, so it could tilt up or face slightly left or right, especially when the Kobuki shook, which would change the readings further.	The ultrasonic sensor could be rigidly mounted at a measured position on the Kobuki, facing straight ahead, and the 575 mm layout threshold and the 410 mm distance target could be recalibrated on the physical robot after mounting. This would improve the accuracy of the layout identification and of the stopping distance, and the reliability of both, since a loose or tilted sensor would no longer change the readings between runs.
3	The Kobuki had little ability to recover from contact in the storage phase. A bumper or cliff trigger in the deposit-distance states sends the Kobuki to the fault state, which stops it and returns it to IDLE, so the run must be restarted by hand. Contact while driving into the storage unit does not cause a fault, but it ends the drive-in early, and the deposit scan then starts from wherever the Kobuki stopped, with no correction for the shorter distance. This is a significant flaw, because the later alignment stages assume the Kobuki has driven in the full distance.	A recovery could be added for contact in the storage phase, such as reversing and rotating away from the wall as in the corridors. For a drive-in that ends early, the Kobuki could check how far it travelled and drive the remaining distance before the scan. This would improve reliability, since a single contact would no longer end the run or leave the scan starting from a poor position.

No.	Limitation	Possible improvement
4	The deposit stage was very slow. No time target was set for it, and it includes a full 360° scan at 50 mm/s followed by a 26-step refine sweep with a 15-tick settling delay and three readings at every step, so it took a large part of the run.	The deposit stage could be sped up by increasing the scan speed, narrowing the sweep or using a larger step, and shortening the settling delay. Each of these trades some accuracy or precision for time, so they would need testing to confirm the alignment is still met.

Table 9 - Limitations of the final design and the improvements that would address them.

Additional Improvement: The alignment could be measured with a protractor, or by measuring the sideways offset between the centre of the Kobuki and the rod, instead of judging it by eye. This would give an exact value and improve the validity of the results by removing the subjective judgement.

8. CONCLUSION

The aim of the project was to configure the Kobuki to handle all four combinations of bollard layout and deposit location using a single program running on the STM32 NUCLEO-G474RE. The final design uses a finite state machine paced by the 50 Hz Kobuki packet, with the layout identified from the range to the first bollard, cliff-sensor-based corridor navigation with wall and ledge recovery, and a coarse-to-fine ultrasonic alignment for the deposit.

In the final test the Kobuki completed both configurations back-to-back, entered the first and second sections within the 30 and 60 second limits, never exceeded 150 mm/s and stopped 106 mm and 102 mm from the deposit rod. All requirements were met (Table 6, Table 7). The main limitations were the number of wall and ledge contacts along the corridors and the lack of recovery during the storage phase (Table 8). Future work should reduce these contacts and shorten the deposit stage while keeping the alignment accuracy.

The aim of the project was to configure the Kobuki to handle all four combinations of bollard layout and deposit location using a single program running on the STM32 NUCLEO-G474RE. The final design uses a finite state machine paced by the 50 Hz Kobuki packet, with the layout identified from the range to the first bollard, cliff-sensor-based corridor navigation with wall and ledge recovery, and a coarse-to-fine ultrasonic alignment for the deposit.

8. REFERENCES

Adelaide University (2026a) *Microcontroller Programming Project: Autonomous Mobile Robot*, project brief, Adelaide University, Adelaide.

Adelaide University (2026b) *Microcontroller Programming Practical 5 Manual: UART and Kobuki bytestream*, Student Edition, Adelaide University, Adelaide.

Adelaide University (2026c) *Microcontroller Programming Practical 6 Manual: FSM, Ultrasonic sensor and Kobuki bytestream*, Student Edition, Adelaide University, Adelaide.

Adelaide University (2026d) *Microcontroller Programming Project Resources: Find object, Drive/Rotate certain Distance/Angle and overall FSM*, Student Edition, Adelaide University, Adelaide.

STMicroelectronics (2026) *STM32G4 Nucleo-64 boards (MB1367)*, User manual UM2505, Rev 7, STMicroelectronics.

Yujin Robot (2017) *Kobuki User Guide*, Version 1.1.0, Yujin Robot.

Yujin Robot (n.d.) *Appendix: Protocol Specification*, Kobuki serial bytestream documentation, Yujin Robot.

10. APPENDICES

The listings below are abridged from the final code. Each caption gives the file the code comes from.

Listing A1: Speed limit saturation in Kobuki_Drive() (kobuki.c).

```
  // Saturation to avoid going over speed limit
if (speed > SPEED_LIMIT) {
    speed = SPEED_LIMIT;
} else if (speed < -SPEED_LIMIT) {
    speed = (~SPEED_LIMIT) + 1;
}
```

Listing A2: Setpoint pattern for a fixed reverse (main.c).

```
// entry block, in change_to()
case STATE_REVERSE:
    Kobuki_Drive_Distance_Setpoint(robot.kobuki.distance, -PATHFIND_REVERSE_MM);
    break;


// state, in the main loop
// Back off a wall or ledge hit.
case STATE_REVERSE:
    if (robot.kobuki.distance_complete != 0) {
        change_to(STATE_ROTATE);
        break;
    }
    Kobuki_Drive_Distance(robot.kobuki.distance);
    break;
```

Listing A3: ADC reading converted to millimetres (ultrasonic.c).

```
uint16_t adc_value = ADC_Read(ULTRASONIC_CHANNEL);
uint32_t distance_mm = ((uint32_t)adc_value * MB1043_FULL_SCALE_MM
                        + (ADC_MAX_VALUE / 2UL)) / ADC_MAX_VALUE;
```

Listing A4: Structure of change_to() (main.c).

```
static void change_to(State next)
{
    switch (state) { ... }   // exit block of the state being left
    state = next;
    switch (next) {          // entry block of the new state
        case STATE_ORIENT:
            Kobuki_Rotate_Angle_Setpoint(robot.kobuki.angle,
                Layout_Pick(ORIENT_TURN_LAYOUT_A_DEG, ORIENT_TURN_LAYOUT_B_DEG));
            break;
        ...
    }
}
```

Listing A5: Layout_From_Range() and Layout_Pick() (helper.c).

```
Layout Layout_From_Range(uint16_t range_mm)
{
    if (range_mm < LAYOUT_B_MAX_RANGE_MM) { return LAYOUT_B; }
    return LAYOUT_A;
}


int16_t Layout_Pick(int16_t a_deg, int16_t b_deg)
{
    return (robot.layout == LAYOUT_A) ? a_deg : b_deg;
}
```

Listing A6: Wrap-safe scan angle in Scan_Update_Turn() (helper.c).

```
int32_t diff = (int32_t)angle_tick - robot.scan_prev_angle;
if (diff > 18000)       { diff -= 36000; }
else if (diff < -18000) { diff += 36000; }
robot.scan_prev_angle = angle_tick;
robot.scan_ticks += diff;
```

Listing A7: Front cliff handling in DRIVE (main.c) and Corridor_Take_Ledge_Hit() (helper.c).

```
if (Kobuki_IsCliffDetected(&robot.kobuki, KOBUKI_SIDE_CENTRE_MASK)) {
    change_to((robot.corridor == 1) ? STATE_CORRIDOR_REVERSE
                                    : STATE_CORRIDOR_STOP);
    break;
}


bool Corridor_Take_Ledge_Hit(const Kobuki_Typedef *kobuki)
{
    // The ledge is on the right in Layout A and on the left in Layout B.
    uint8_t ledge_mask = (robot.layout == LAYOUT_A) ? KOBUKI_SIDE_RIGHT_MASK
                                                    : KOBUKI_SIDE_LEFT_MASK;
    if (!Kobuki_IsCliffDetected(kobuki, ledge_mask)) { return false; }
    robot.pathfind_recover_deg = (int16_t)(-Layout_Pick(
        PATHFIND_RECOVER_TURN_LAYOUT_A_DEG, PATHFIND_RECOVER_TURN_LAYOUT_B_DEG));
    return true;
}
```

Listing A8: Raw cliff threshold in Kobuki_Read() (kobuki.c).

```
if (kobuki_rx_buffer[CLIFF_ID] == 0x05U) {
    for (uint8_t s = 0U; s < 3U; s++) {
        kobuki->cliff_raw[s] = (uint16_t)kobuki_rx_buffer[CLIFF_RAW + (2U * s)] |
            (uint16_t)((uint16_t)kobuki_rx_buffer[CLIFF_RAW + (2U * s) + 1U] << 8);
        if (kobuki->cliff_raw[s] < CLIFF_RAW_LIMIT) {
            kobuki->cliff |= (uint8_t)(1U << s);
        }
    }
}
```

Listing A9: Nearest valid echo in Deposit_Scan_Sample() (helper.c).

```
if (range_mm > 0 && range_mm <= DEPOSIT_SCAN_MAX_RANGE_MM &&
    range_mm < robot.deposit_rod.range_mm) {
    robot.deposit_rod.detected = 1;
    robot.deposit_rod.range_mm = range_mm;
    robot.deposit_rod_angle = robot.kobuki.angle;
}
```

Listing A10: Median stored for each refine step in Deposit_Refine_Sample() (helper.c).

```
// the three readings are sorted; the middle one is the median
refine->ranges[refine->point] = refine->readings[1];
refine->angles[refine->point] = robot.kobuki.angle;
```

Listing A11: Distance adjustment decision in DEPOSIT_DISTANCE (main.c).

```
if (robot.deposit_range_mm <
    DEPOSIT_TARGET_RANGE_MM - DEPOSIT_RANGE_TOLERANCE_MM) {
    change_to(STATE_DEPOSIT_REVERSE);
} else if (robot.deposit_range_mm >
           DEPOSIT_TARGET_RANGE_MM + DEPOSIT_RANGE_TOLERANCE_MM) {
    change_to(STATE_DEPOSIT_APPROACH);
} else {
    Kobuki_Update_LEDs(GREEN, OFF);
    Win_Sound_Start();
    change_to(STATE_IDLE);
}
```

Listing A12: Fault_B1_Wheeldrop() and Fault_Any() (helper.c).

```
// Safety checks used to stop the robot when a button, wheel drop, bumper, or cliff is detected.

bool Fault_B1_Wheeldrop(void)

{

    return (robot.kobuki.button & KOBUKI_BUTTON_B1_MASK) != 0 ||

           robot.kobuki.wheeldrop != 0;

}



bool Fault_Any(void)

{

    return Fault_B1_Wheeldrop() || robot.kobuki.bumper != 0 ||

           Kobuki_IsCliffDetected(&robot.kobuki, KOBUKI_SIDE_ALL_MASK);

}
```

Listing A13: Win sound table (helper.c) and note conversion in Kobuki_Play_Note() (kobuki.c).

```
static const WinSoundNote win_sound[] = {

    {NOTE_F_7, 100, 60},      // beep

    {NOTE_F_7, 100, 60},      // beep

    ...

    {NOTE_C_8, 180, 0},

};



// in Kobuki_Play_Note(): the note value is 363636 / frequency, rounded

uint16_t note = (uint16_t)((SOUND_NOTE_NUMERATOR + (freq_hz / 2U)) / freq_hz);
```

Search and Orientation Flowchart

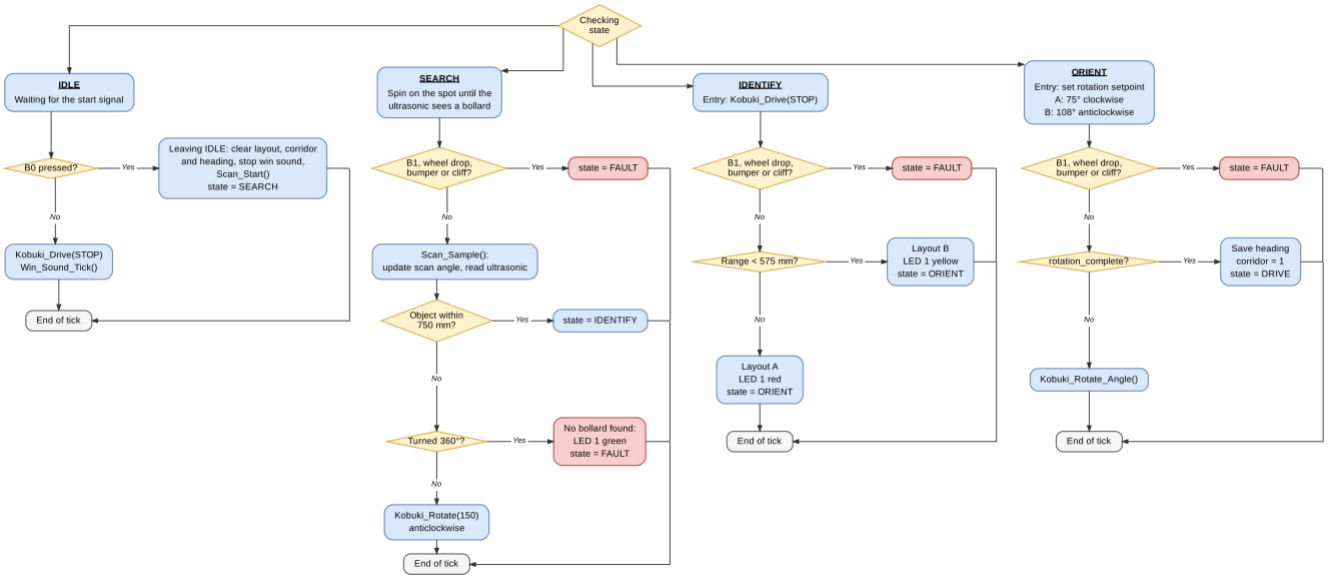


Figure B1 – Search and orientation flowchart. The Kobuki spins to find the first bollard, identifies the layout from its range and turns to face corridor 1.

Pathfind Flowchart

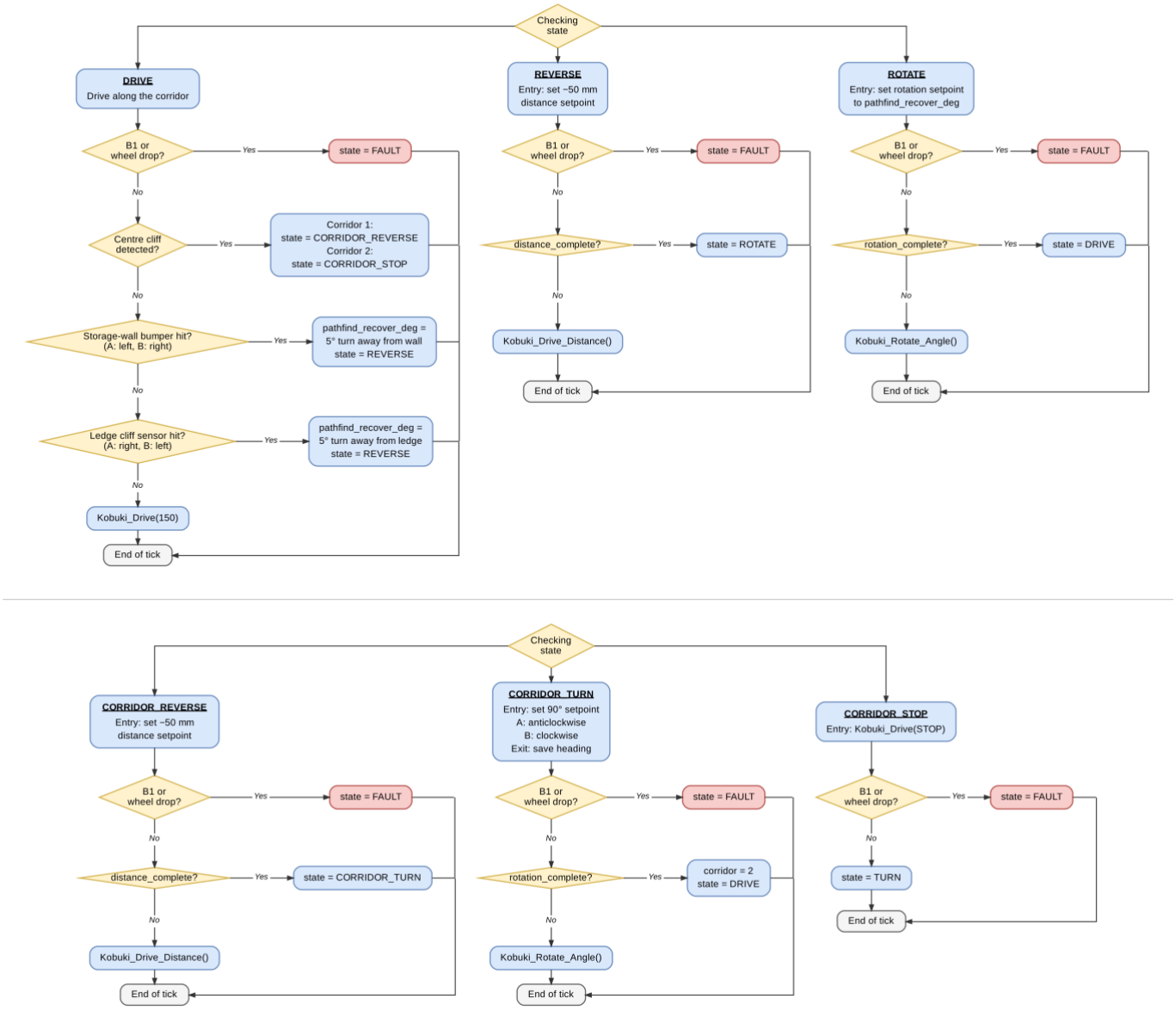


Figure B2 – Pathfind flowchart. The Kobuki drives each corridor and recovers from wall and ledge contact. The centre cliff sensor ends each corridor, and the Kobuki then turns the corner or stops at the storage unit.

Storage Flowchart

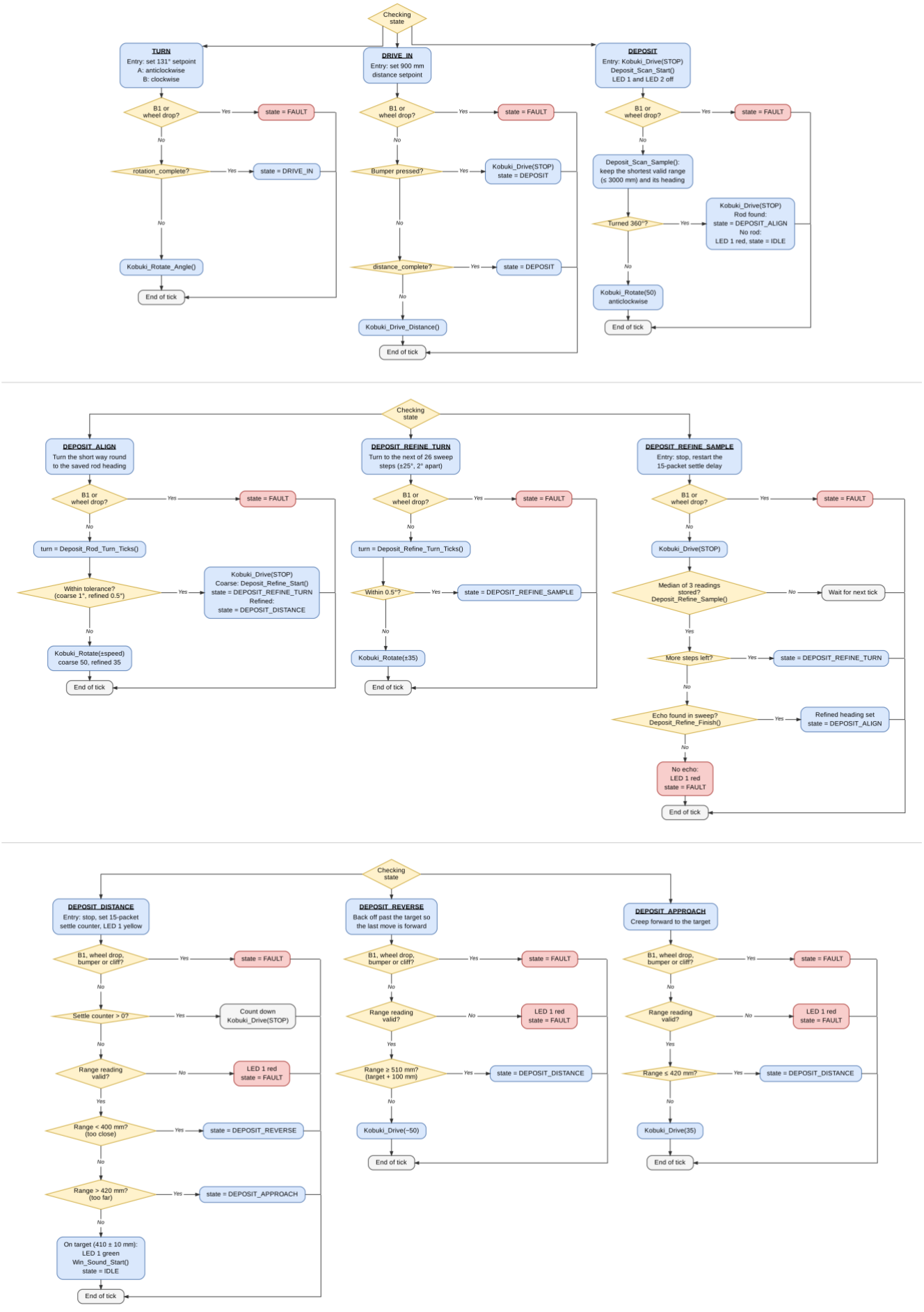


Figure B3 – Storage flowchart. The Kobuki turns into the storage unit, drives in, scans for the rod and aligns to it. It then refines the heading with a local sweep and adjusts to the final distance.